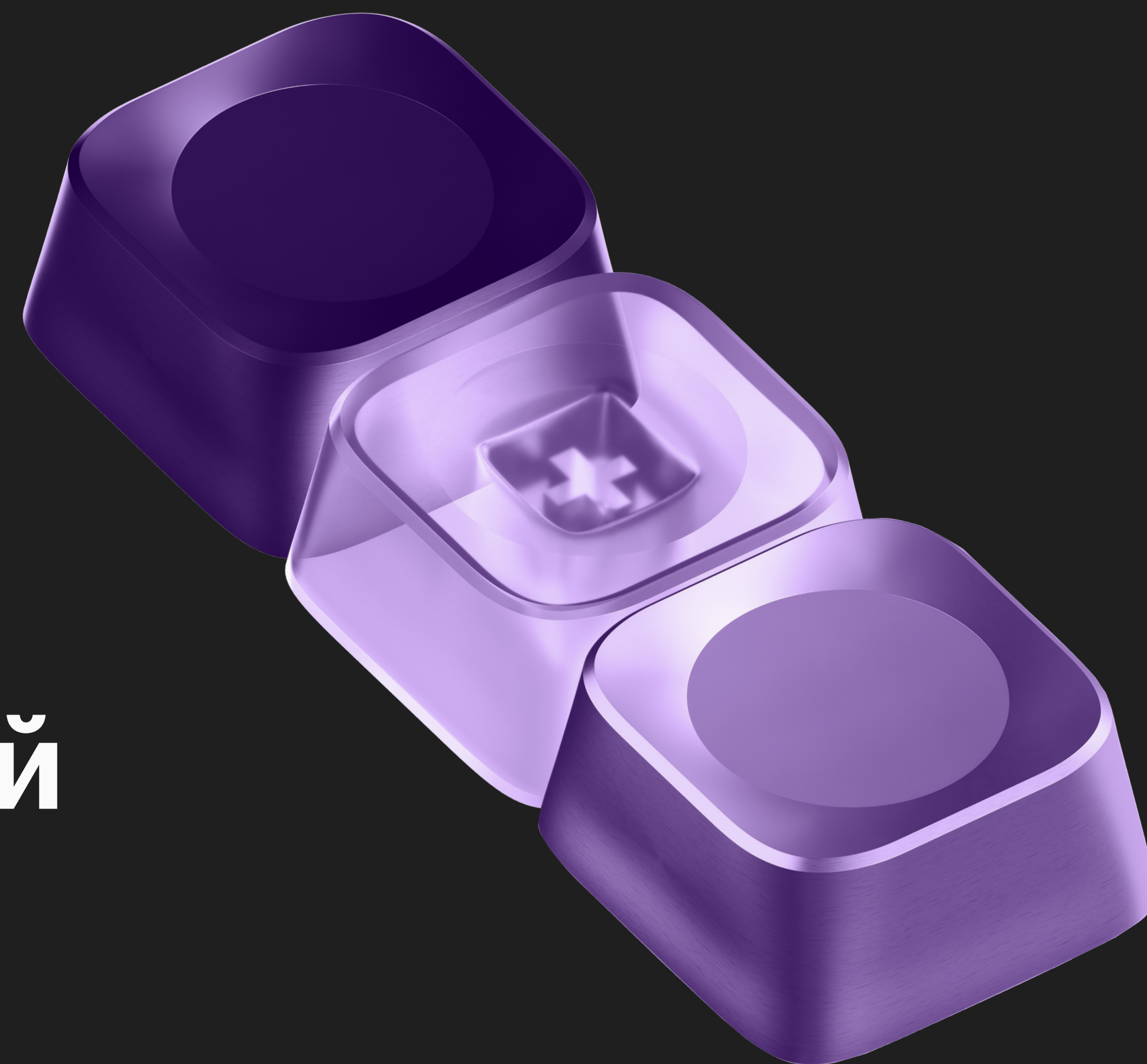


Транзакции. Уровни изоляции транзакций

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 11



План лекции

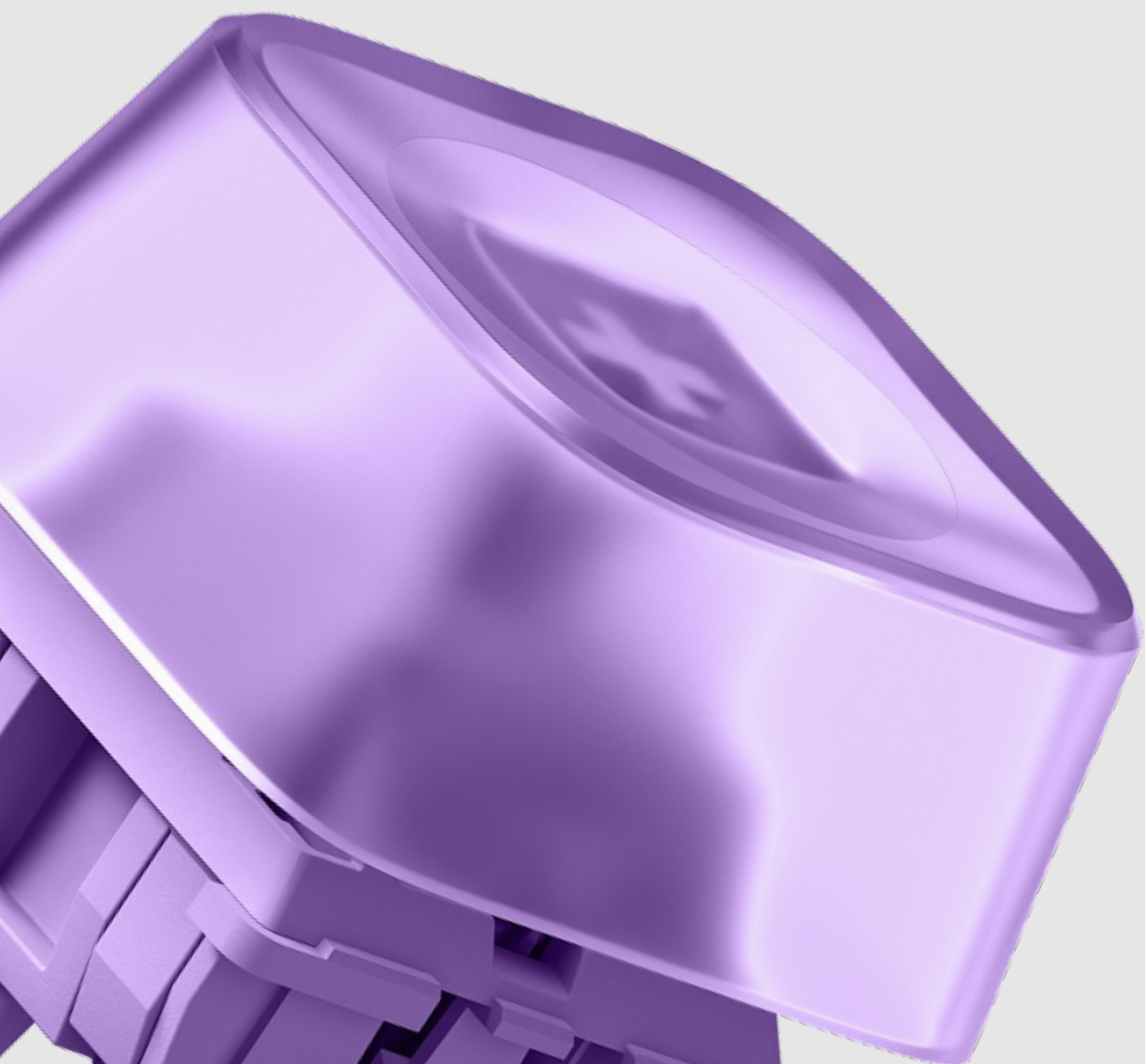
→ Фундамент и проблема параллелизма

→ Аномалии и стандарты

→ MVCC в PostgreSQL

→ Уровни изоляции в реалиях Postgres

→ Транзакции и надёжность

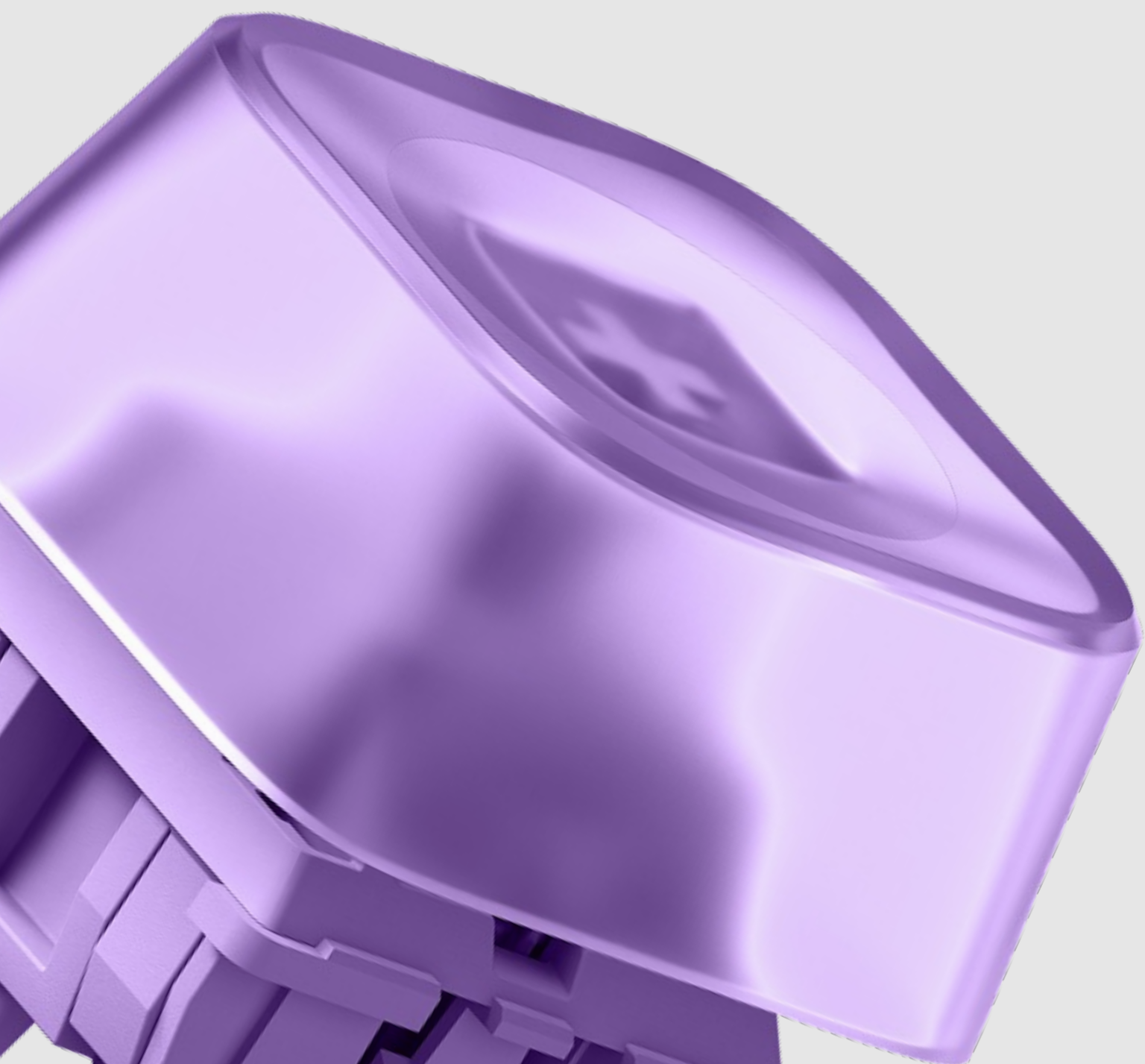


Контракт ACID

→ Atomicity, Consistency, Durability — бинарные гарантии. Они либо есть, либо их нет.

→ Isolation — это не константа, а спектр компромиссов между консистентностью данных и пропускной способностью системы.

→ Идеальная изоляция = нулевой параллелизм.



Serial vs Interleaved Execution

- Serial Execution: безопасно по определению. Исключает любые конфликты доступа.
- Interleaved Execution: необходимость для утилизации CPU/IO и обеспечения приемлемого Latency при высоких RPS.
- Фундаментальная проблема: чередование операций с общим состоянием неминуемо порождает аномалии конкурентного доступа.

Serial Schedule

Transaction T1	Transaction T2
R(A)	
W(A)	
R(B)	
W(B)	
commit	
	R(A)
	W(B)
	commit

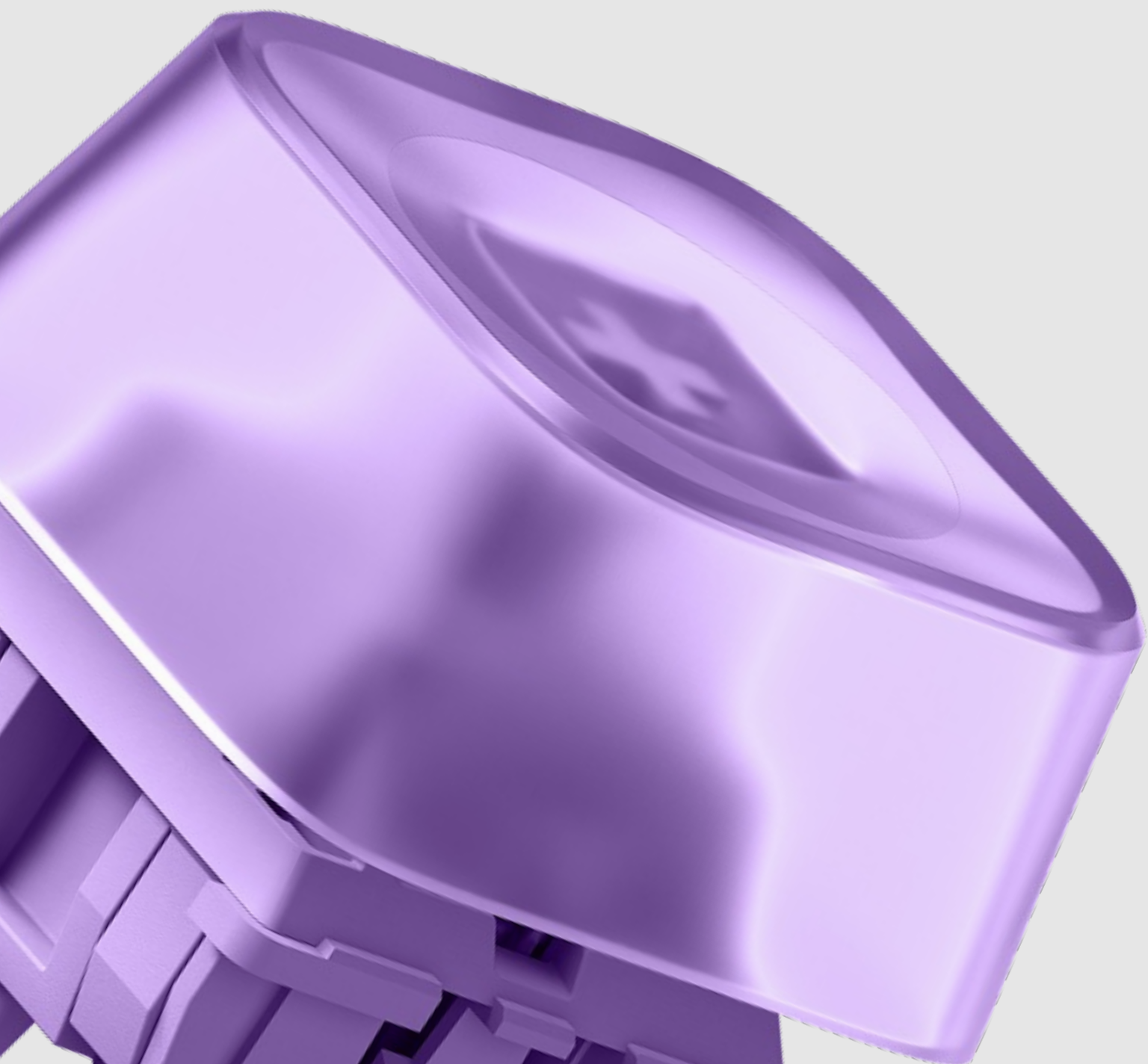
- Maximum safety
- Zero concurrency
- High Latency

Interleaved Schedule

Transaction T1	Transaction T2
R1(A)	
W1(A)	
	R2(A)
	W2(A)
R1(B)	
W1(B)	
	R2(B)
	W2(B)

- High throughput
- Risk of anomalies

Архитектурные парадигмы Concurrency Control



- Pessimistic (Lock-based/2PL): «Блокируй заранее». Строгая сериализация через графы ожиданий. Проблема: deadlocks и низкая скорость чтения.
- Optimistic (OCC): «Делай что хочешь, проверим при коммите». Нет блокировок в процессе. Проблема: массовые rollback при высокой конкуренции за запись.
- Multi-Version (MVCC): «Каждому своя версия истории». Читатели читают старое, писатели создают новое. Проблема: требует сборки мусора (Garbage Collection) и доп. памяти.

Стандарт ANSI SQL-92

Table 3. ANSI SQL Isolation Levels Defined in terms of the four phenomena				
Isolation Level	P 0 Dirty Write	P 1 Dirty Read	P 2 Fuzzy Read	P 3 Phantom
READ UNCOMMITTED	Not Possible	Possible	Possible	Possible
READ COMMITTED	Not Possible	Not Possible	Possible	Possible
REPEATABLE READ	Not Possible	Not Possible	Not Possible	Possible
SERIALIZABLE	Not Possible	Not Possible	Not Possible	Not Possible

Dirty Read (Грязное чтение / P1)

1. Чтение незафиксированных (uncommitted) изменений другой транзакции.
2. Фундаментальное нарушение изоляции: работа с данными, которых «никогда не существовало».

*Начальное значение salary = 900

T1

T2

```
UPDATE salary = 1000
```

```
SELECT salary  
-> salary = 1000
```

```
ROLLBACK
```

```
UPDATE ... SET  
salary*0.10  
-> salary = 1100
```


Non-repeatable vs Phantom Read

1. Non-repeatable Read (P2):
изменение конкретного кортежа (Tuple). UPDATE/DELETE.
2. Phantom Read (P3): изменение
математического множества
строк, удовлетворяющих условию
(Predicate). INSERT/DELETE.

T1

```
SELECT salary FROM ...  
WHERE id=1  
-> salary=1000
```

T2

```
UPDATE ... SET salary=1000  
WHERE id=1 + COMMIT
```

```
SELECT salary ... WHERE  
id=1  
-> salary=1000
```

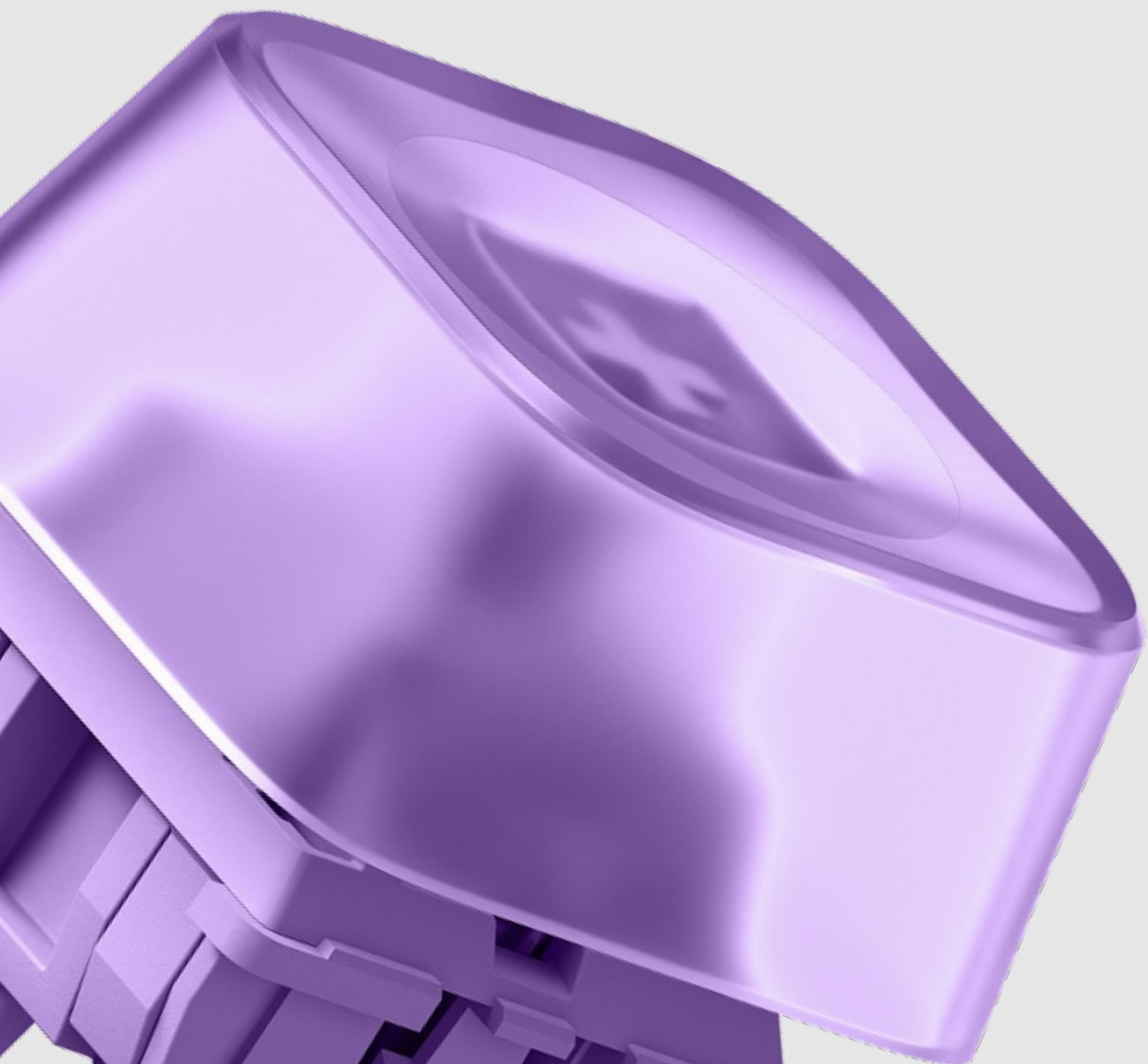
```
SELECT count(id) FROM ...  
WHERE role='admin'  
->10
```

```
INSERT INTO ... (role)  
VALUES ('admin') + COMMIT
```

```
SELECT count(id) FROM ...  
WHERE role='admin'  
->11
```


Критика SQL-92

- Стандарт SQL-92 слеп к аномалиям, возникающим в MVCC.
- Необходимость введения новых феноменов: Lost Update (P4) и Write Skew (A5B)
- Доказательство того, что уровни изоляции — это не линейная шкала, а сложный граф зависимостей.



Современные аномалии

Потерянное обновление
(Lost Update)

1. Две транзакции читают одно и то же значение.
2. Обе обновляют его на основе прочитанного.
3. Второй COMMIT затирает первый.

T1

T2

```
SELECT tickets=1
```

```
SELECT tickets=1
```

```
UPDATE tickets=0
```

```
UPDATE tickets=0
```

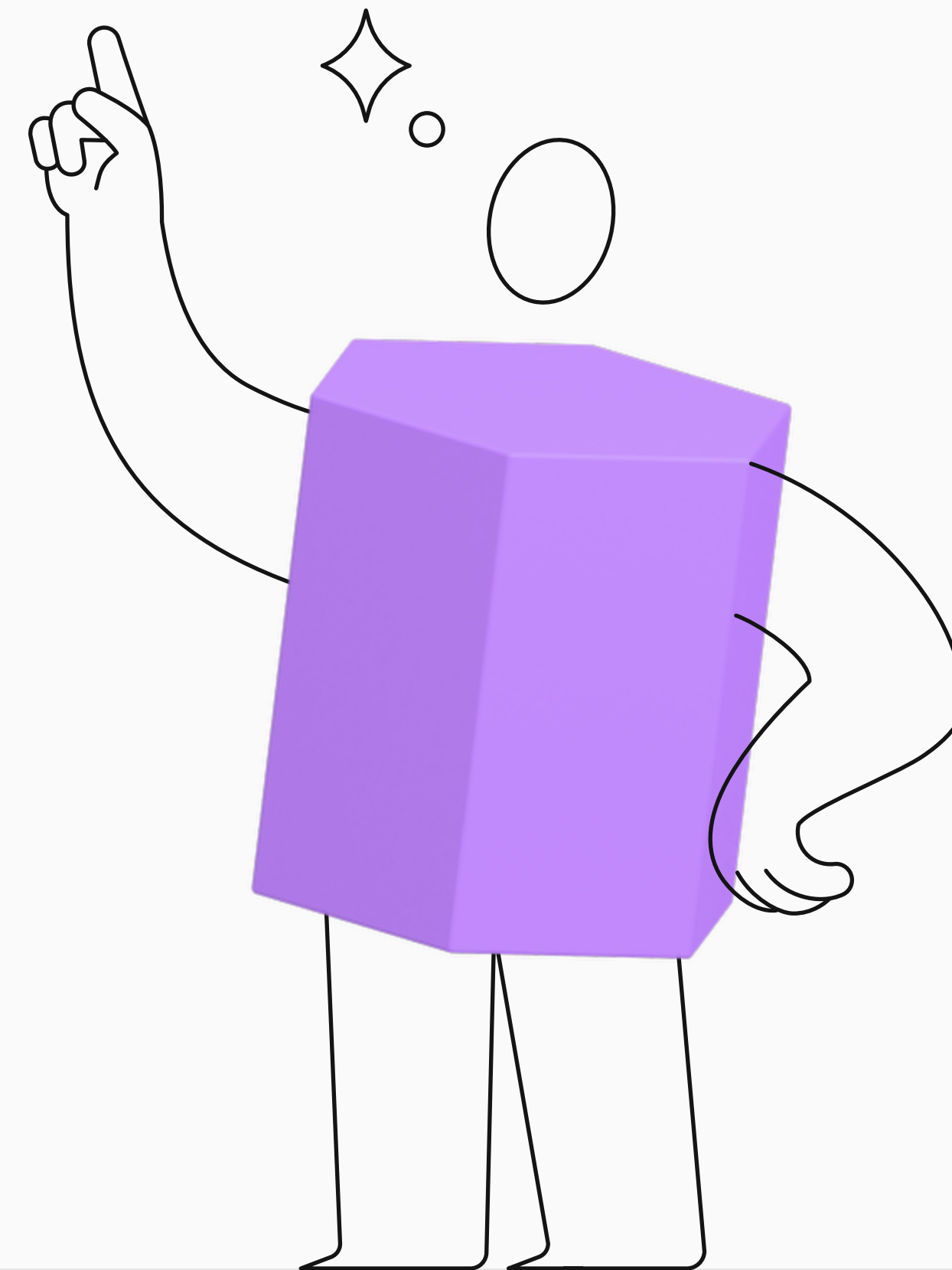
Современные аномалии

Перекус записи (Write Skew)

Правило: на дежурстве всегда должен быть хотя бы 1 врач (Alice или Bob).

- **T1 (Alice):** видит, что Bob дежурит → делает UPDATE ... set status='off'.
- **T2 (Bob):** видит, что Alice дежурит → делает UPDATE ... set status='off'.

Итог: оба ушли, в больнице 0 дежурных.



Стандарт vs PostgreSQL



Стандарт SQL

	Грязное чтение	Неповторяющееся чтение	Фантомное чтение	Другие аномалии
Read Uncommitted	да	да	да	да
Read Committed	нет	да	да	да
Repeatable Read	нет	нет	да	да
Serializable	нет	нет	нет	нет



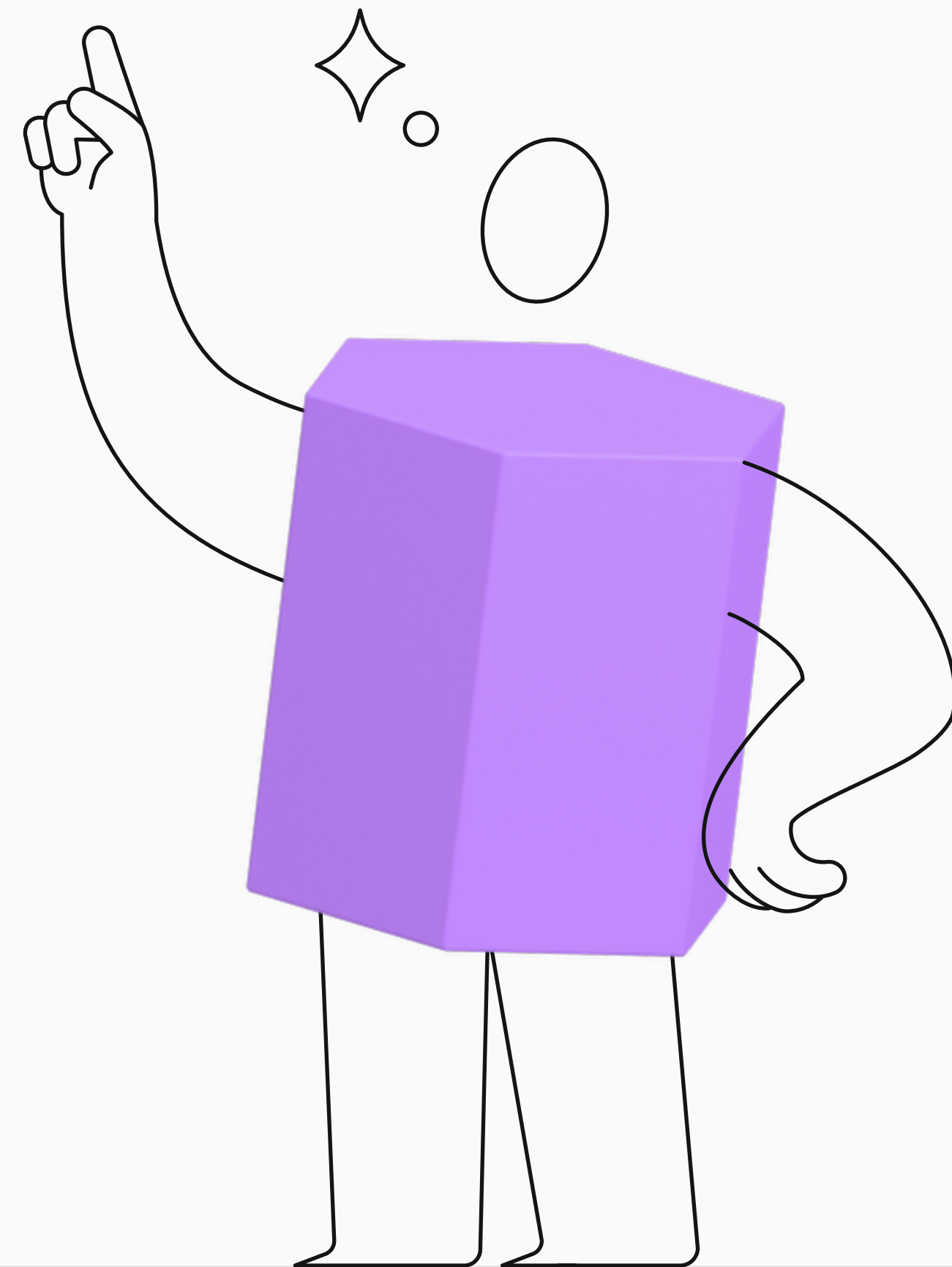
PostgreSQL

	Грязное чтение	Неповторяющееся чтение	Фантомное чтение	Другие аномалии
Read Uncommitted	нет	да	да	да
Read Committed	нет	да	да	да
Repeatable Read	нет	нет	нет	да
Serializable	нет	нет	нет	нет

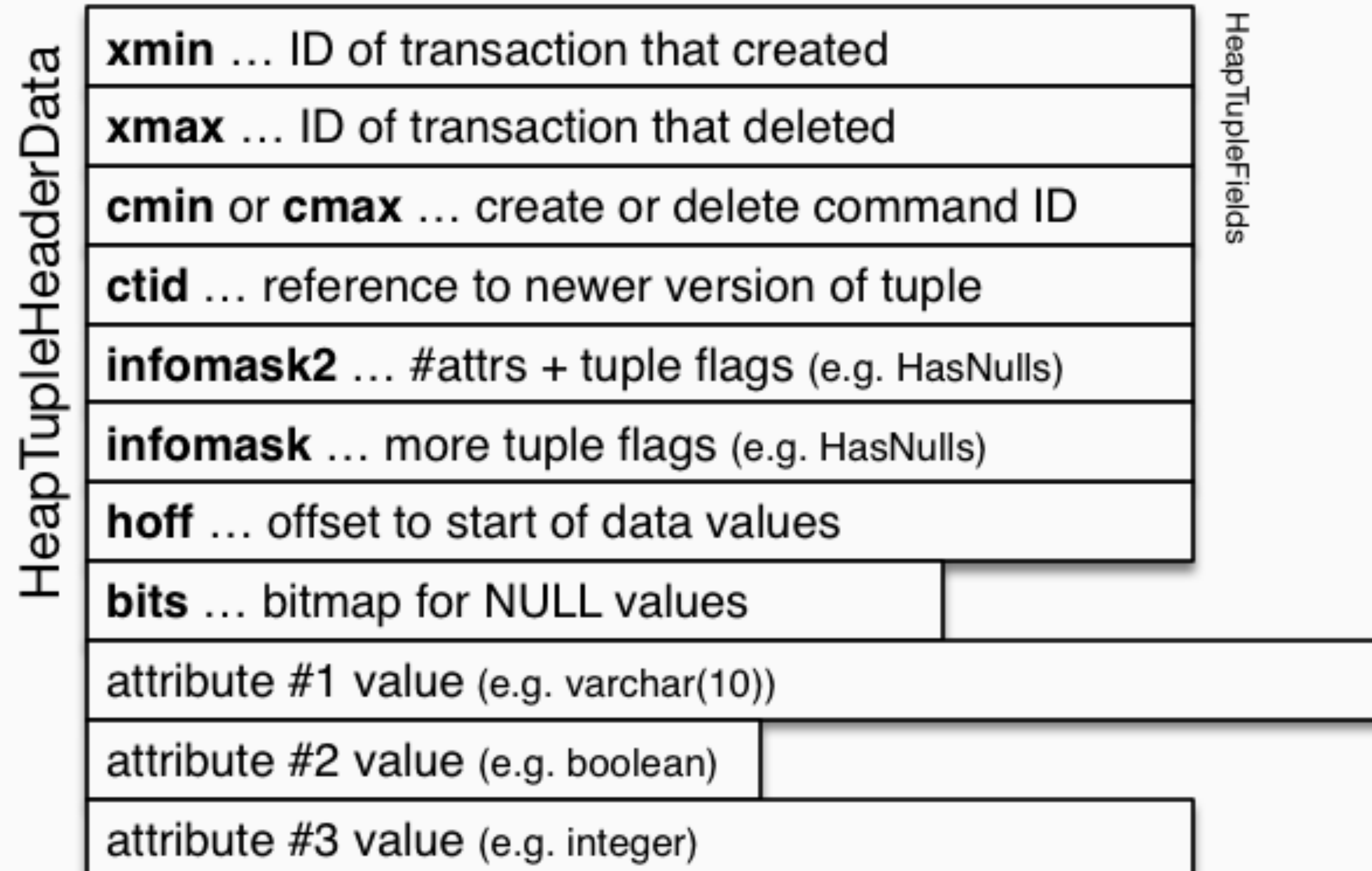
MVCC. Пространство вместо времени

Lock-based — строка заблокирована, очередь транзакций ждёт.

MVCC — транзакция порождает новую версию строки рядом со старой. Быстро, но расходует дисковое пространство.

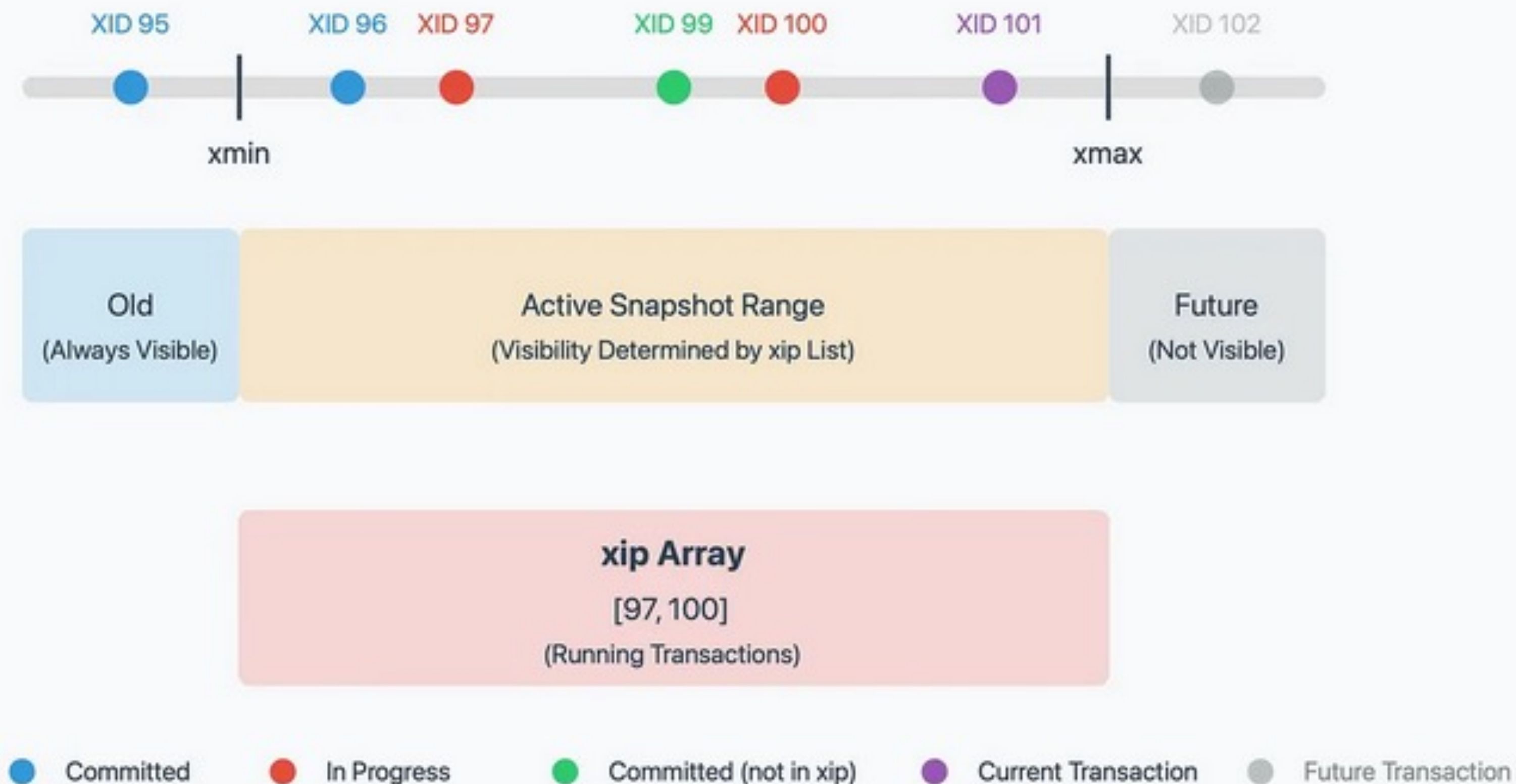


Заголовок кортежа



Глобальный журнал статусов

pg_xact (ex-CLOG)



* pg_xact — структура данных, хранящая статус всех когда-либо запускавшихся транзакций.

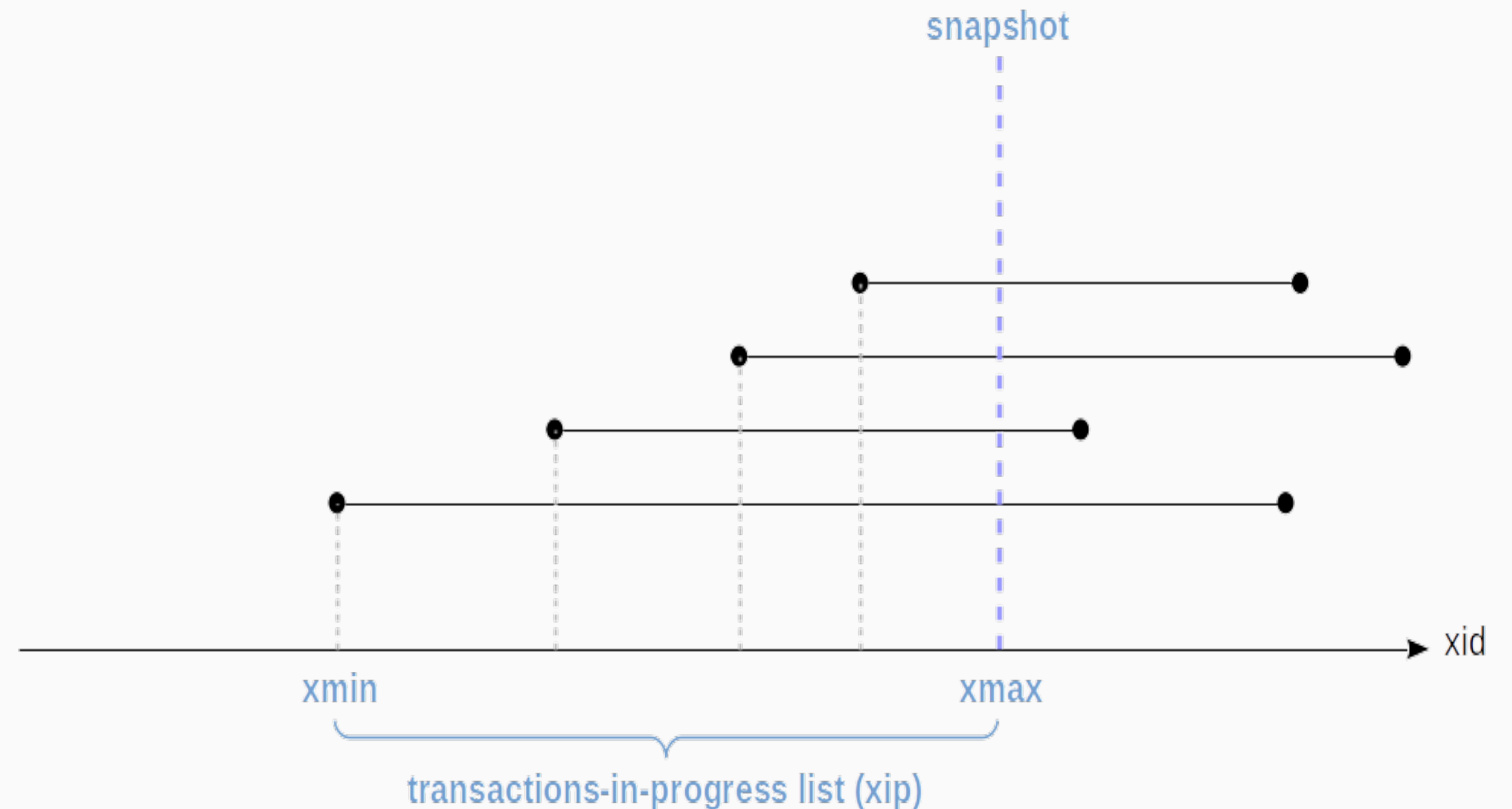
Оптимизация видимости

Hint Bits

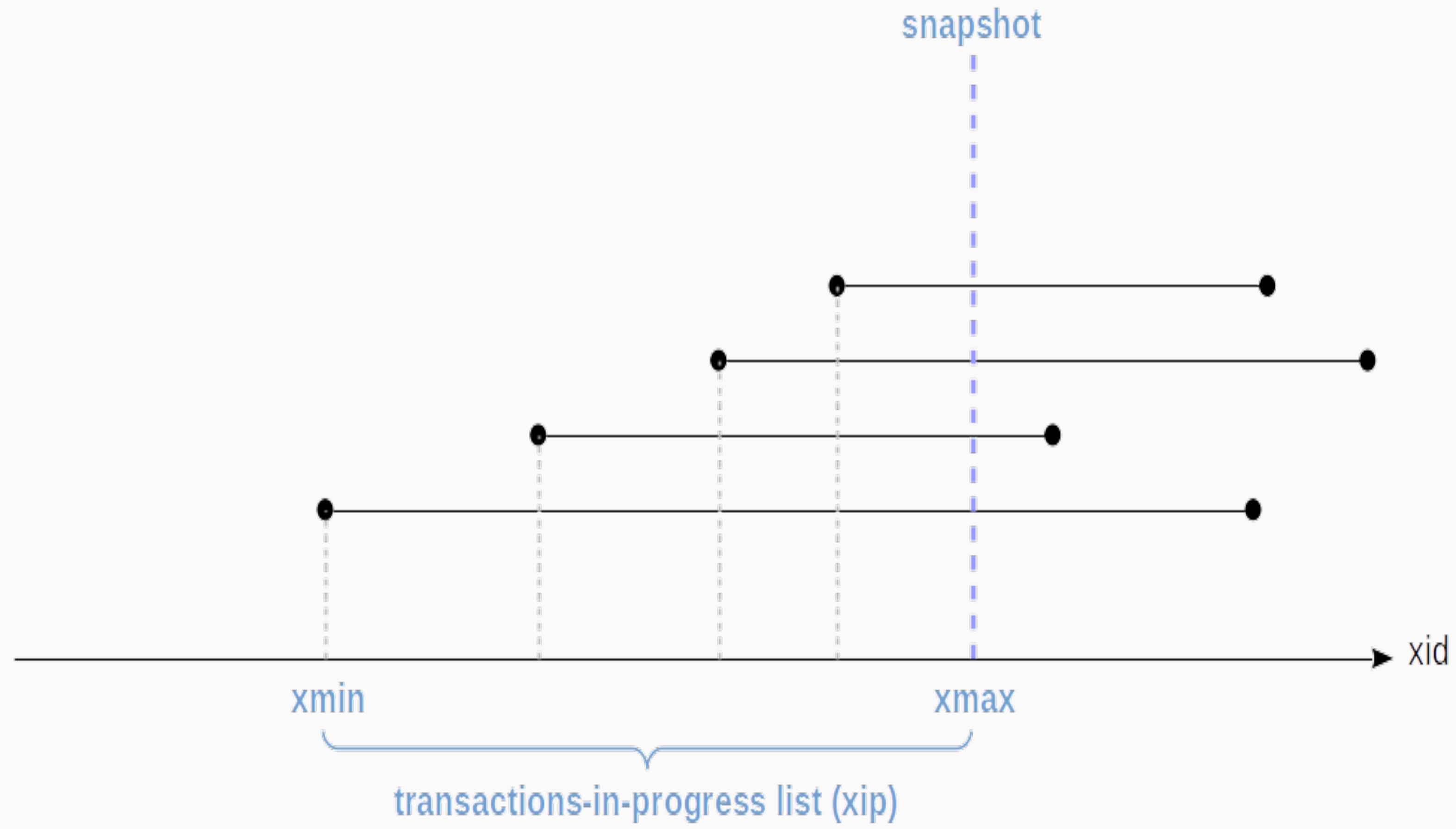
```
• • •  
  
#define HEAP_XMIN_COMMITTED      0x0100    /* t_xmin committed */  
#define HEAP_XMIN_INVALID        0x0200    /* t_xmin invalid/aborted */  
#define HEAP_XMAX_COMMITTED      0x0400    /* t_xmax committed */  
#define HEAP_XMAX_INVALID        0x0800    /* t_xmax invalid/aborted */
```

Транзакционные снимки

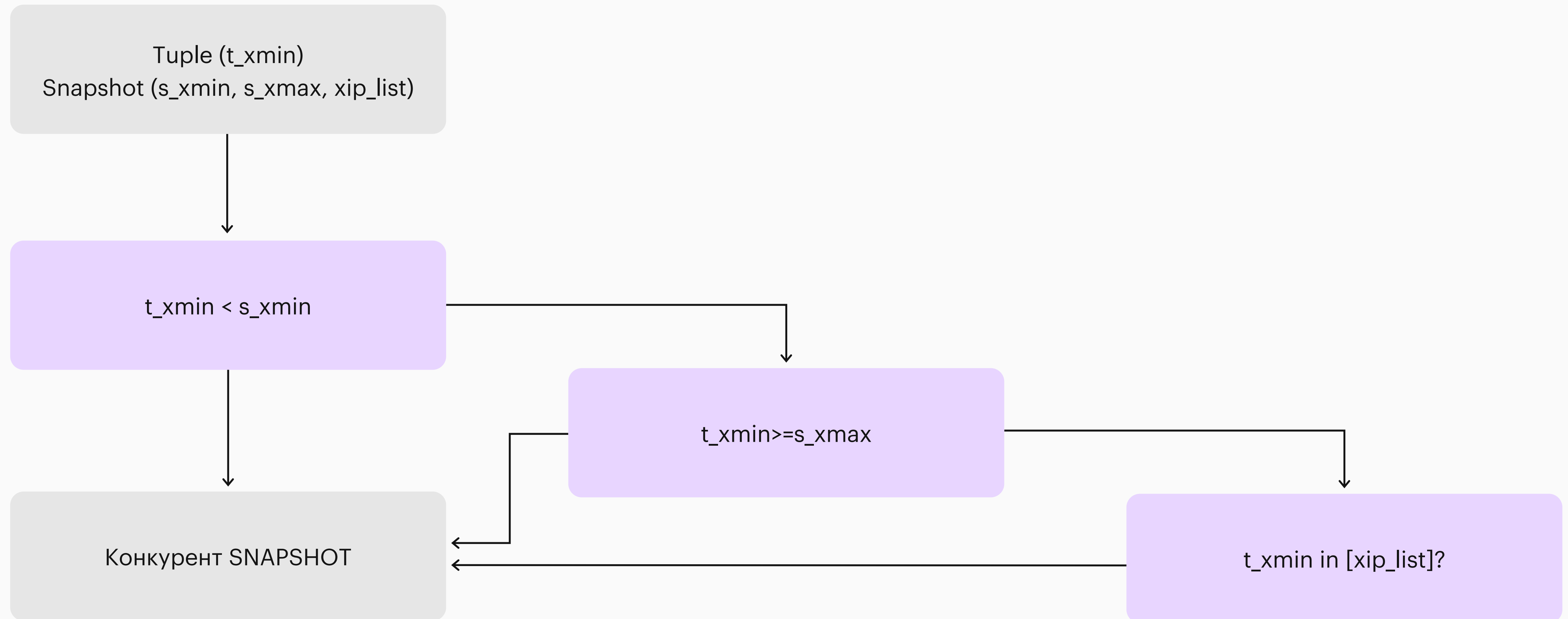
- Знать, что x_{min} закоммичен, недостаточно для обеспечения изоляции.
- Вопрос не в том, закоммичен ли он вообще, а в том, был ли он закоммичен в момент старта моего запроса?
- Snapshot — это слепок состояния конкурентной среды в конкретный микросекундный момент.



Snapshot



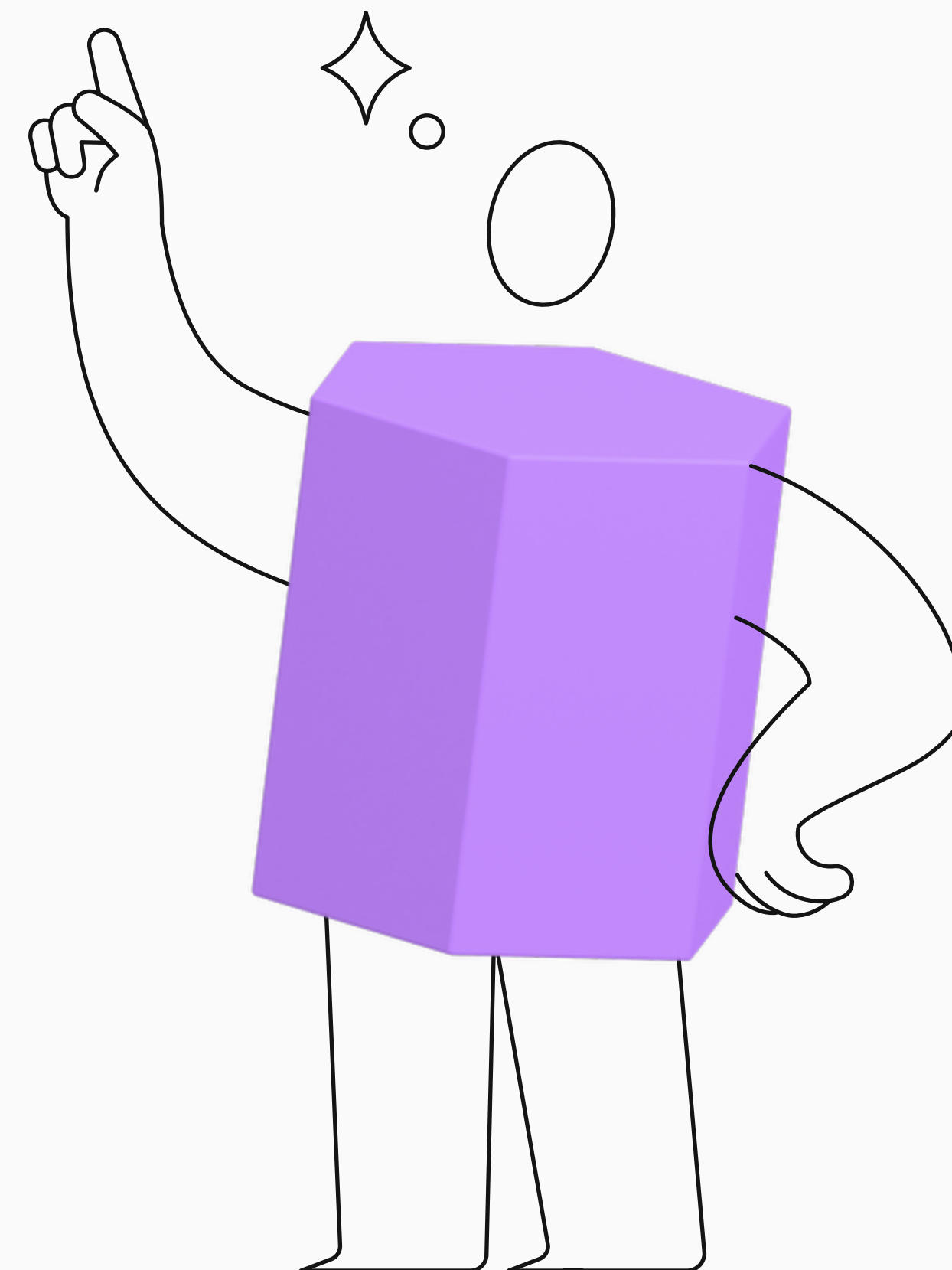
Visibility Rules Engine



Уровни изоляции

Физическая реализация

- **Read Committed** — перед каждым SELECT делается вызов функции GetSnapshotData().
- **Repeatable Read** — функция GetSnapshotData() вызывается только один раз при выполнении первого оператора **SELECT**.
- **Уровни изоляции в MVCC** — это просто правила обновления снимка (Snapshot).
- RC видит новые коммиты, потому что делает новый снимок на каждый запрос.
- RR живёт в «пузыре» времени старта первой команды.



Транзакционные снимки

```
postgres=# BEGIN ISOLATION LEVEL SERIALIZABLE;
BEGIN
postgres=# SELECT * FROM inventory WHERE id=1;
 id | name  | qty
----+-----+----
  1 | active | 100
(1 row)

postgres=# UPDATE inventory SET qty=110 WHERE id=1;
UPDATE 1
postgres=# COMMIT;
COMMIT
postgres=#
```

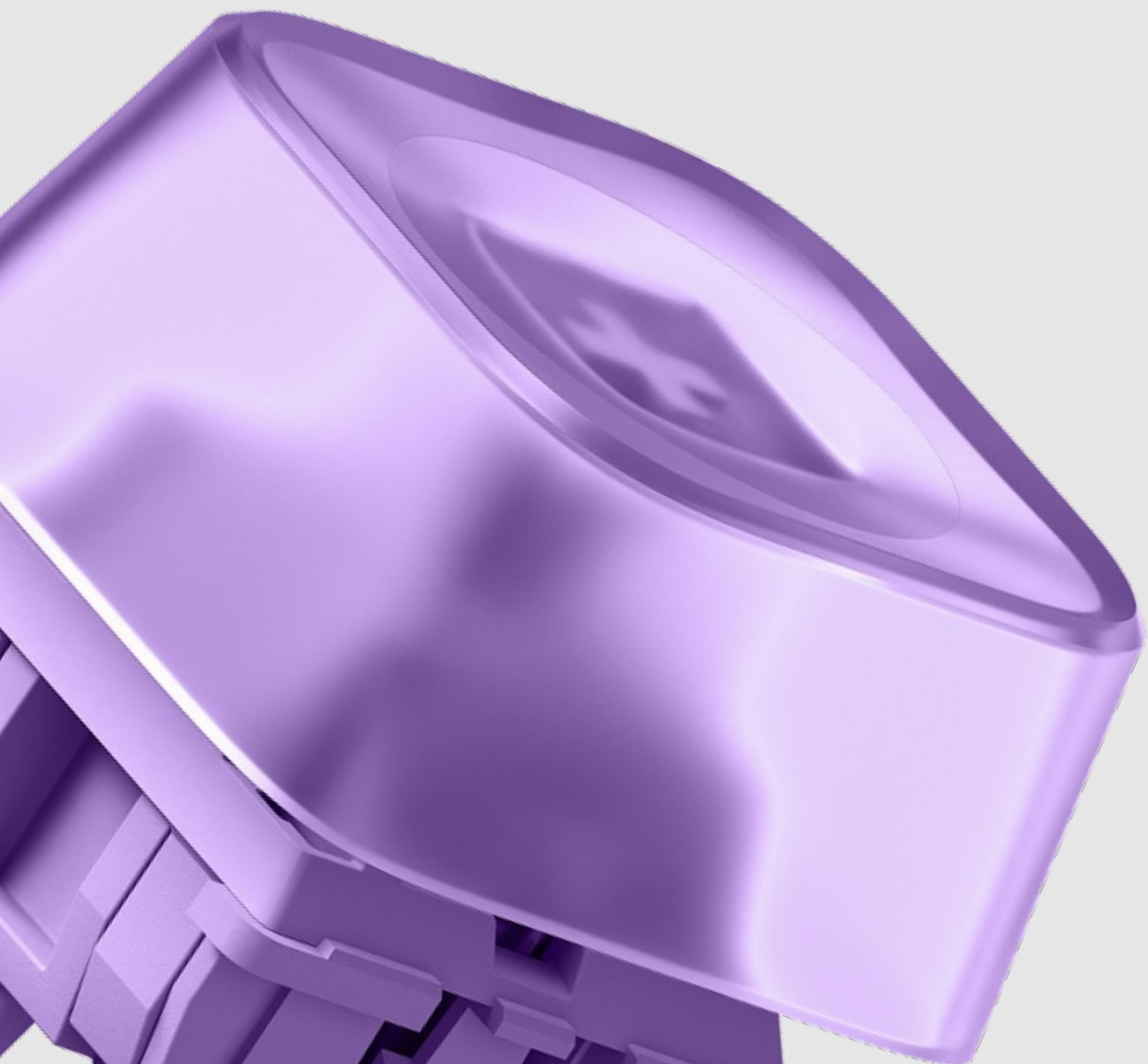
```
postgres=# BEGIN ISOLATION LEVEL SERIALIZABLE;
BEGIN
postgres=# SELECT * FROM inventory WHERE id=1;
 id | name  | qty
----+-----+----
  1 | active | 100
(1 row)

postgres=# UPDATE inventory SET qty=120 WHERE id=1;
ERROR:  could not serialize access due to concurrent update
postgres=#
```

Решение: Добавить блок try/catch или rescue, который бы перехватывал состояние SQLSTATE 40001 и делал бы retry для транзакции.

MVCC

Garbage Collection

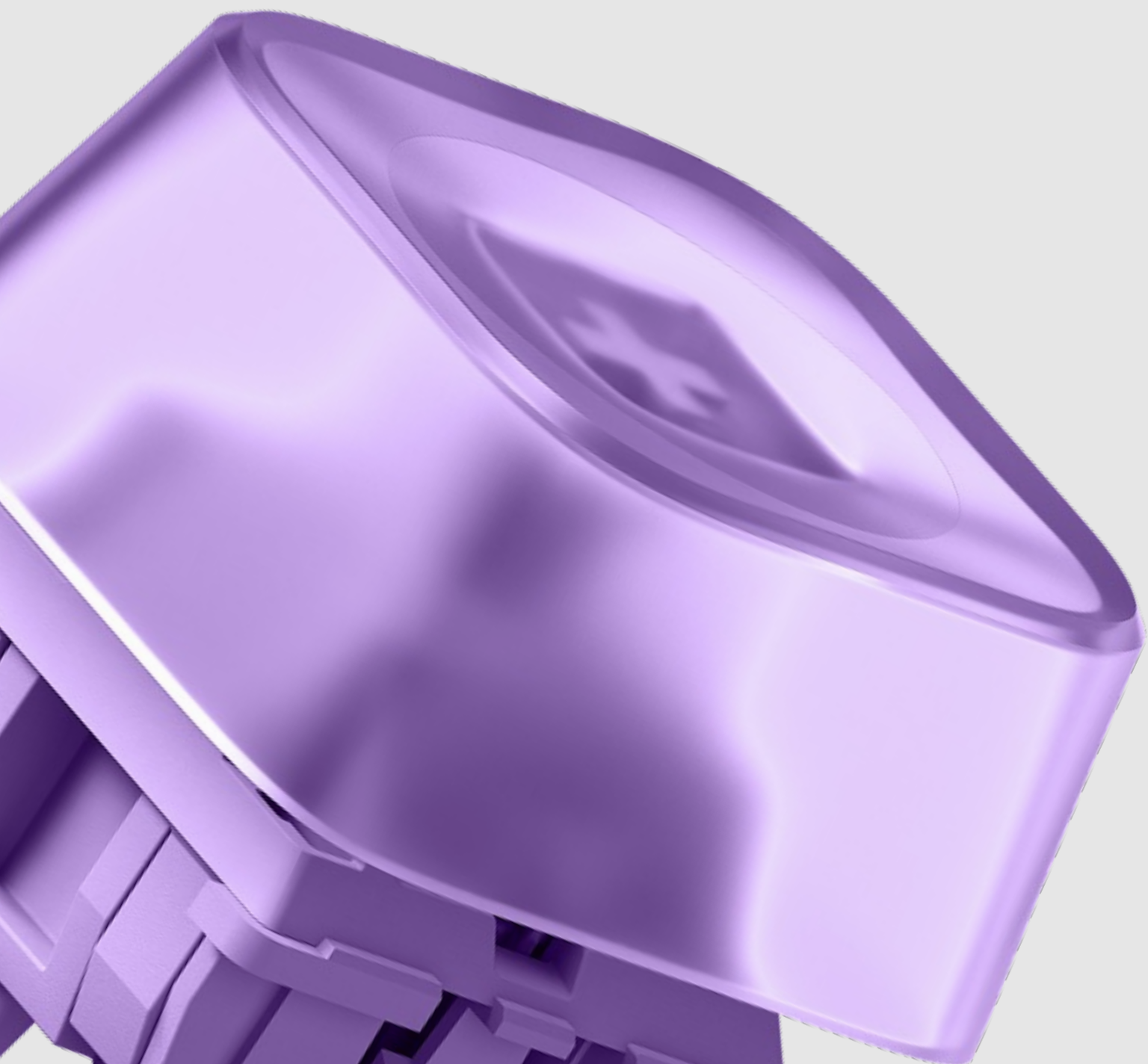


→ Старые версии строк накапливаются и раздувают физические файлы таблиц и индексов (Table/Index Bloat).

→ Мёртвые кортежи — строки, которые не видны ни одному активному снепшоту.

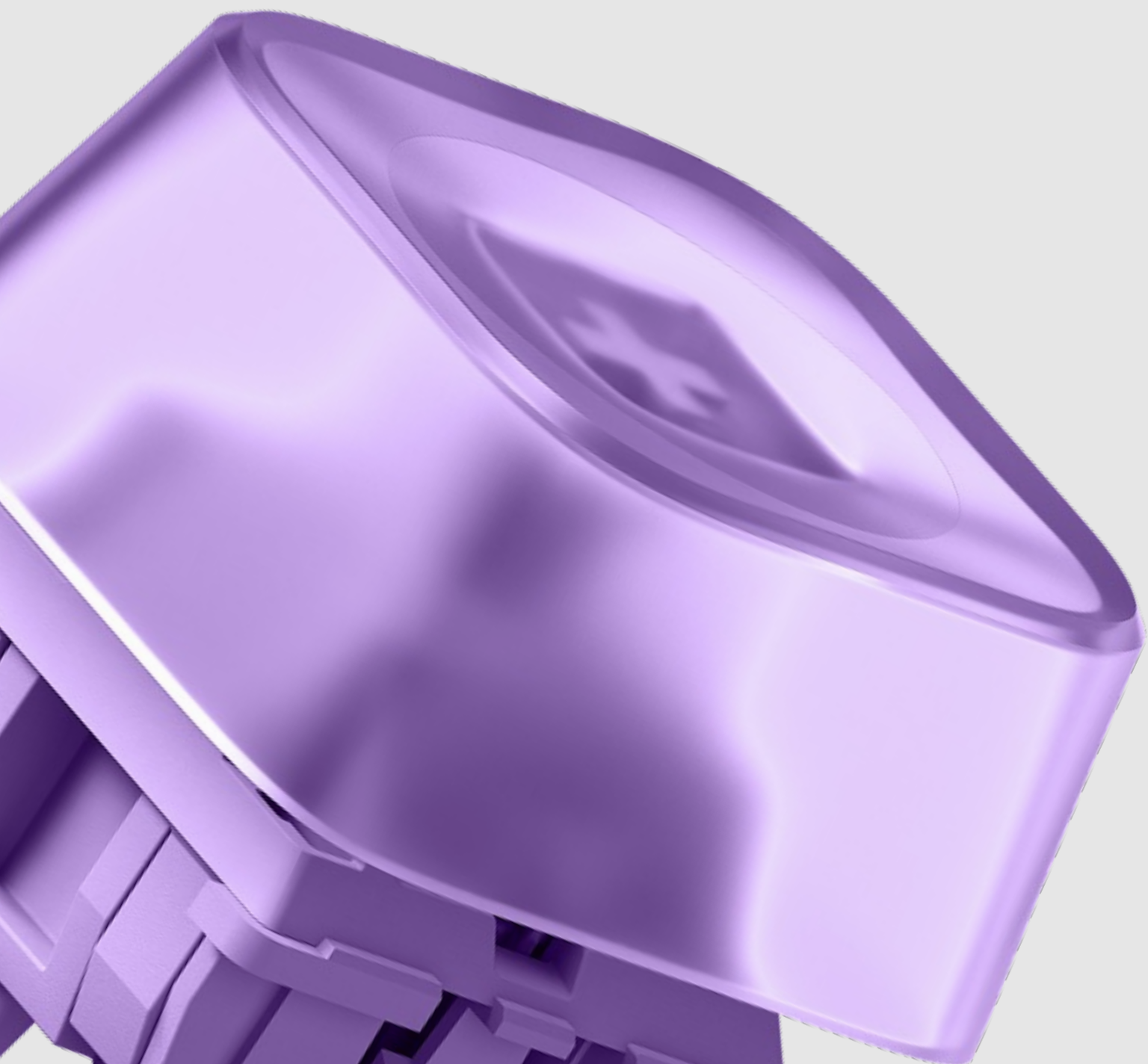
→ Фоновый процесс autovacuum — критический компонент системы. Сканирует таблицы, помечает мёртвые строки как свободное.

Read Uncommitted



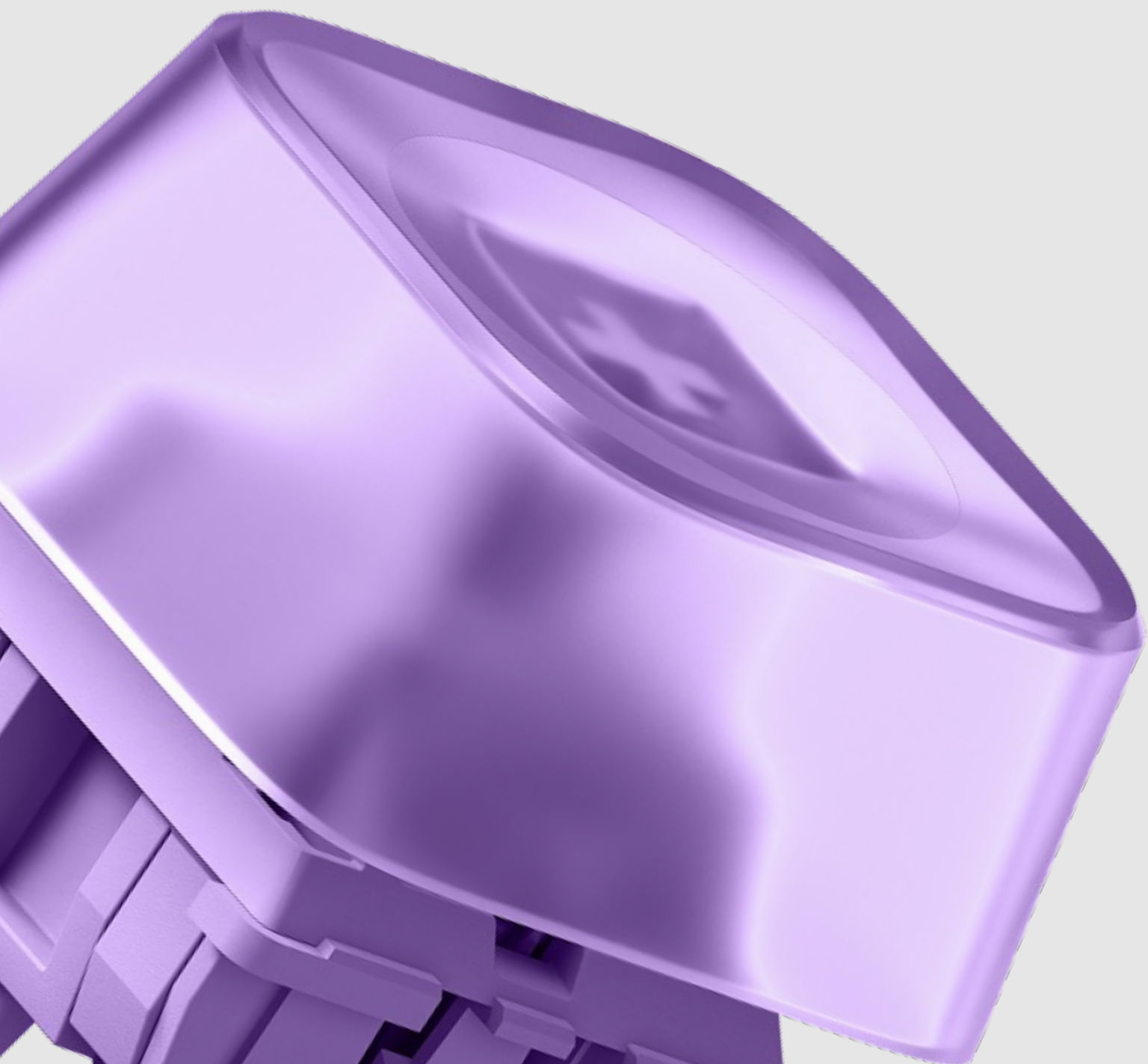
- В PostgreSQL уровень Read Uncommitted физически не существует как отдельный механизм.
- Архитектура MVCC делает «грязное чтение» невозможным (строка с незавершённым `t_xmin` отбрасывается правилами видимости снимка).
- Стандарт разрешает СУБД предоставлять более строгий уровень изоляции, чем запросил пользователь.

Read Committed



- Базовый уровень изоляции в Postgres.
- Snapshot создаётся заново на каждый отдельный оператор (statement).
- Идеально для быстрых коротких транзакций, но смертельно для сложных аналитических отчётов (Non-repeatable read).

Repeatable Read



→ Снимок фиксируется ровно один раз — на первой инструкции транзакции.

→ В PostgreSQL этот уровень защищает от Phantom Read.

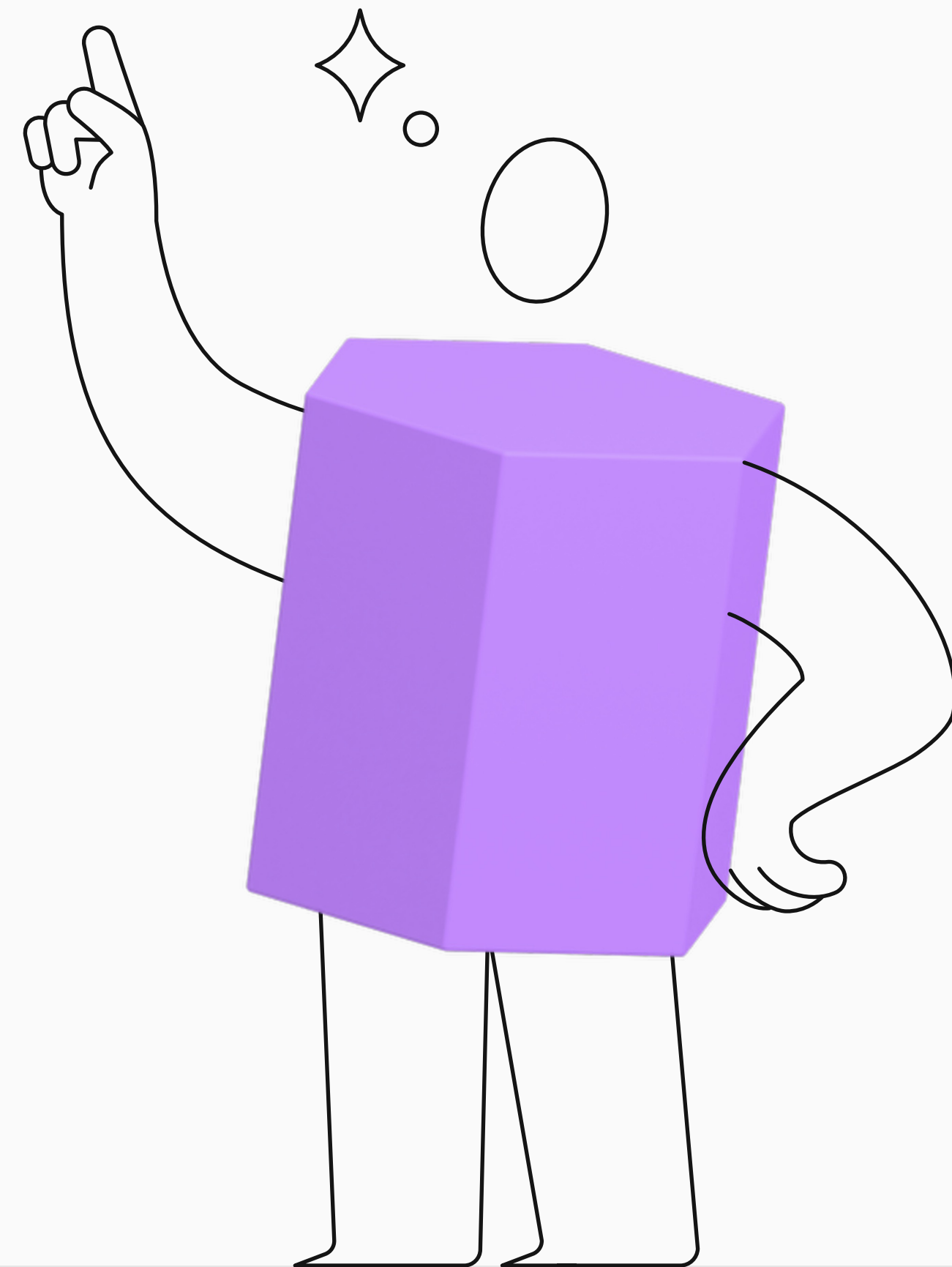
→ Цена: First-committer wins. Если вы попытаетесь обновить строку, которую уже обновил конкурент, вы получите сериализационный сбой.

Слепое пятно MVCC

Конфликт дежурных врачей

- **А:** обновляет строку id=1 (создавая версию id=1_v2).
- **В:** обновляет строку id=2 (создавая версию id=2_v2).

Результатом обе транзакции закоммитятся, но в этом случае бизнес-логика нарушена, так как на дежурстве опять 0 врачей.

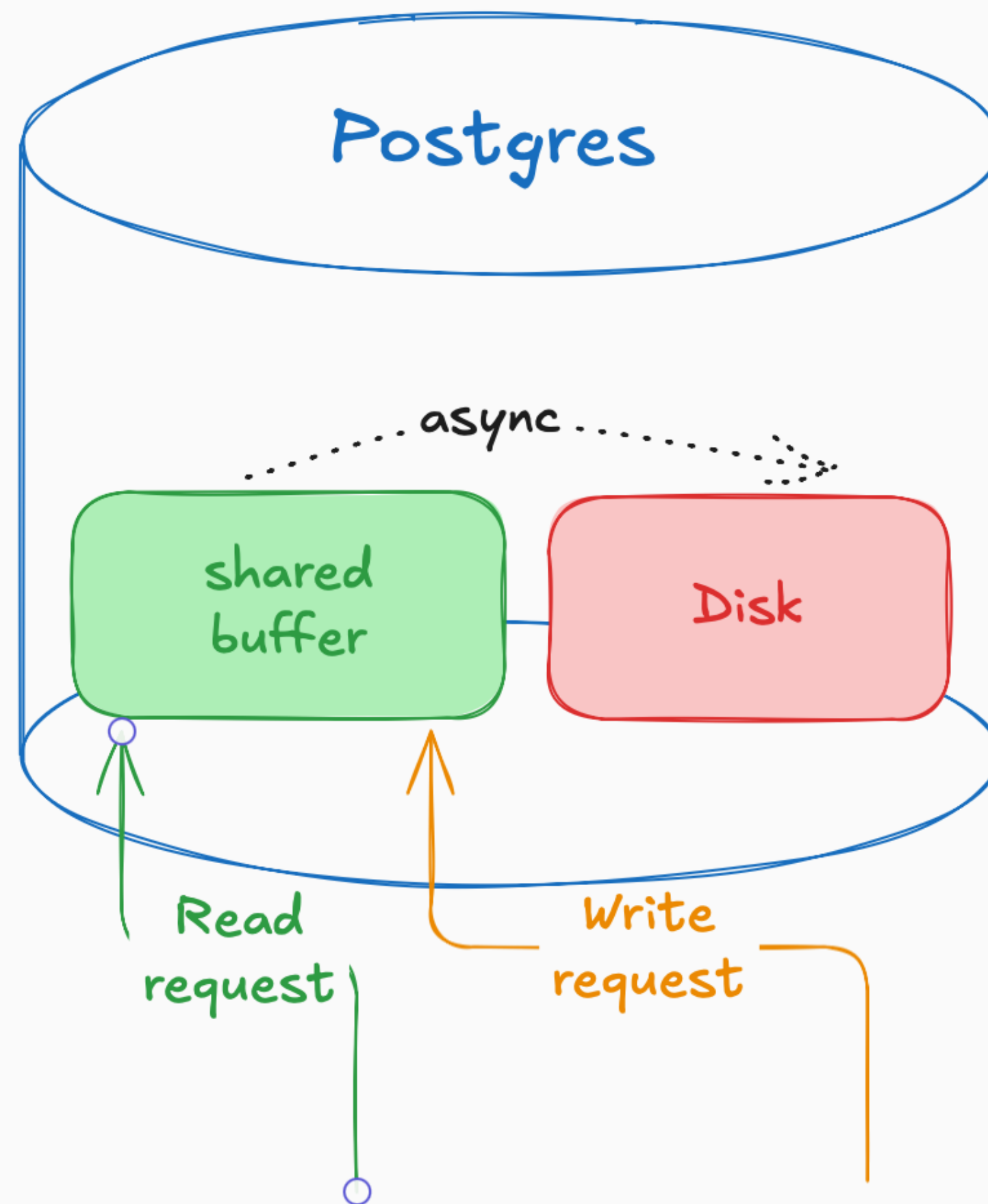


Serializable

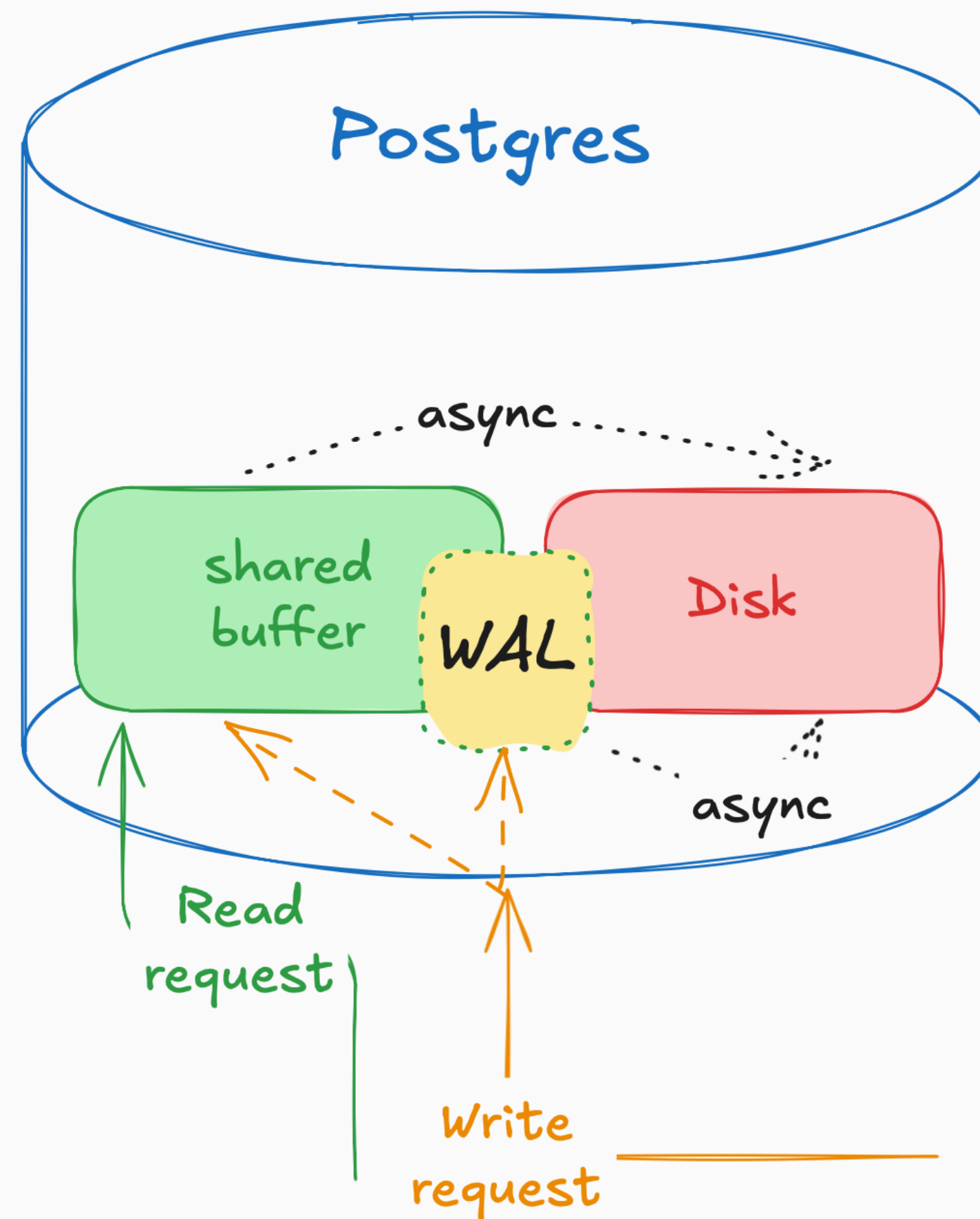
SSI (Serializable Snapshot Isolation)



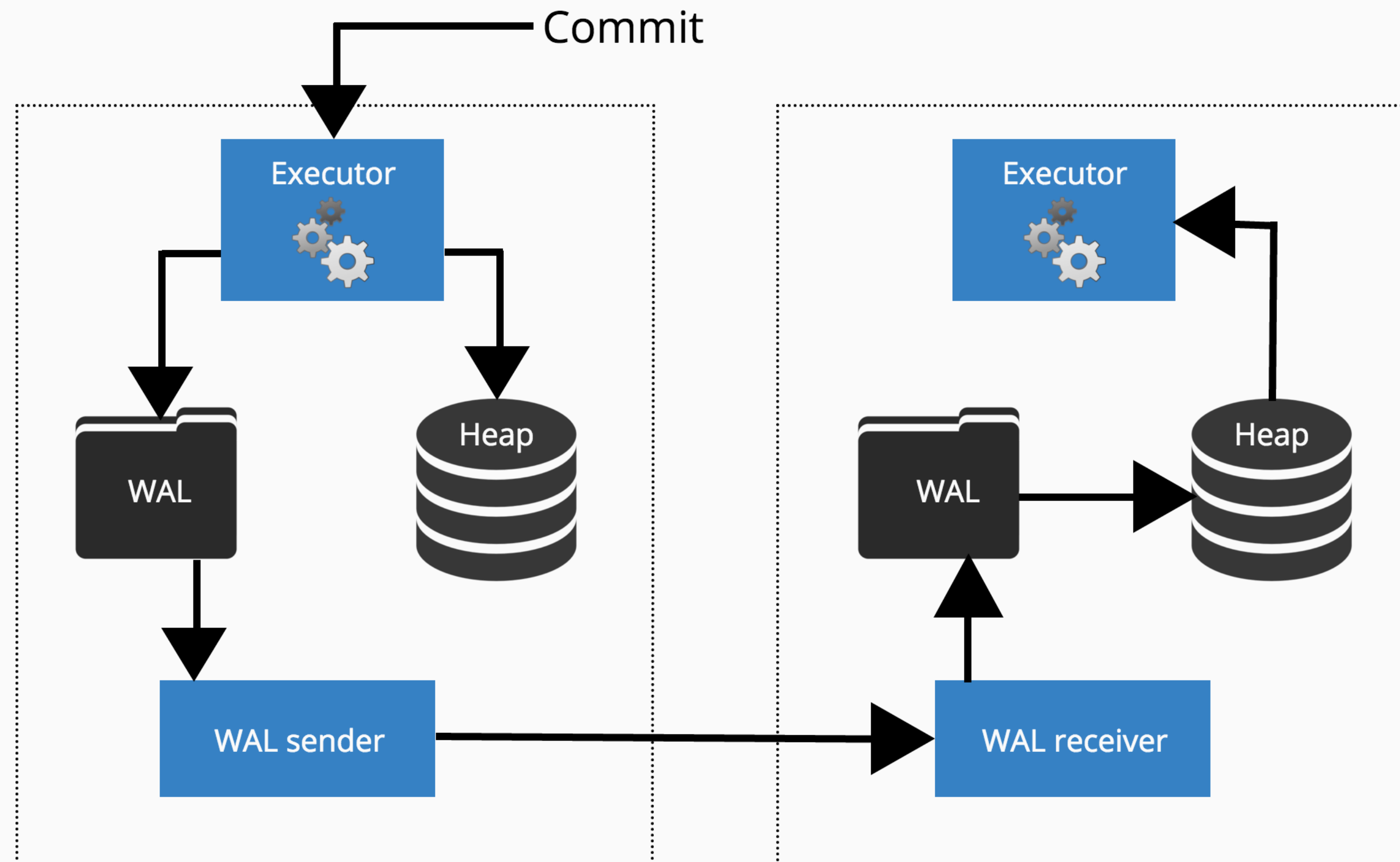
I/O-операции



Write-Ahead Log (WAL)

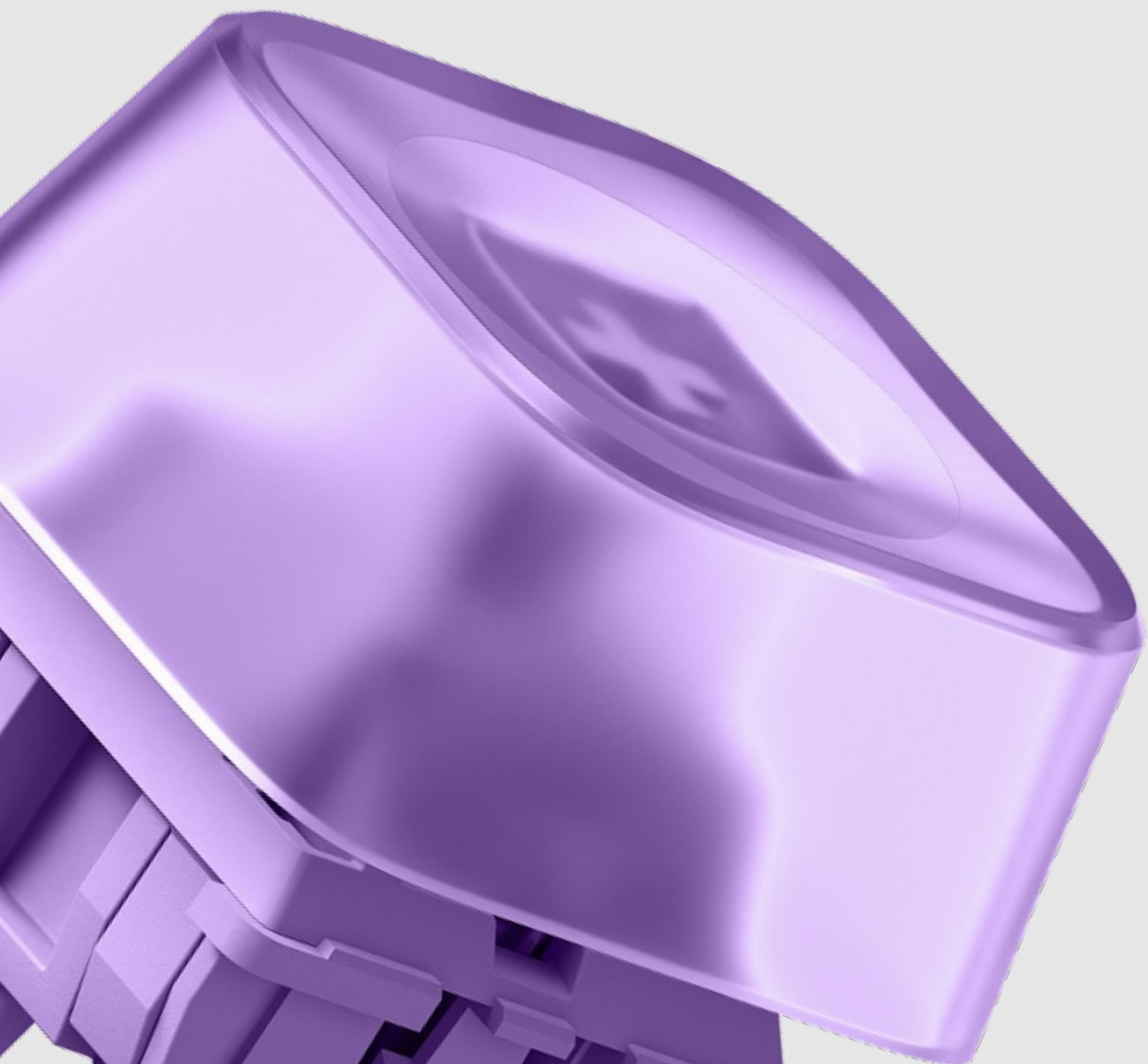


COMMIT

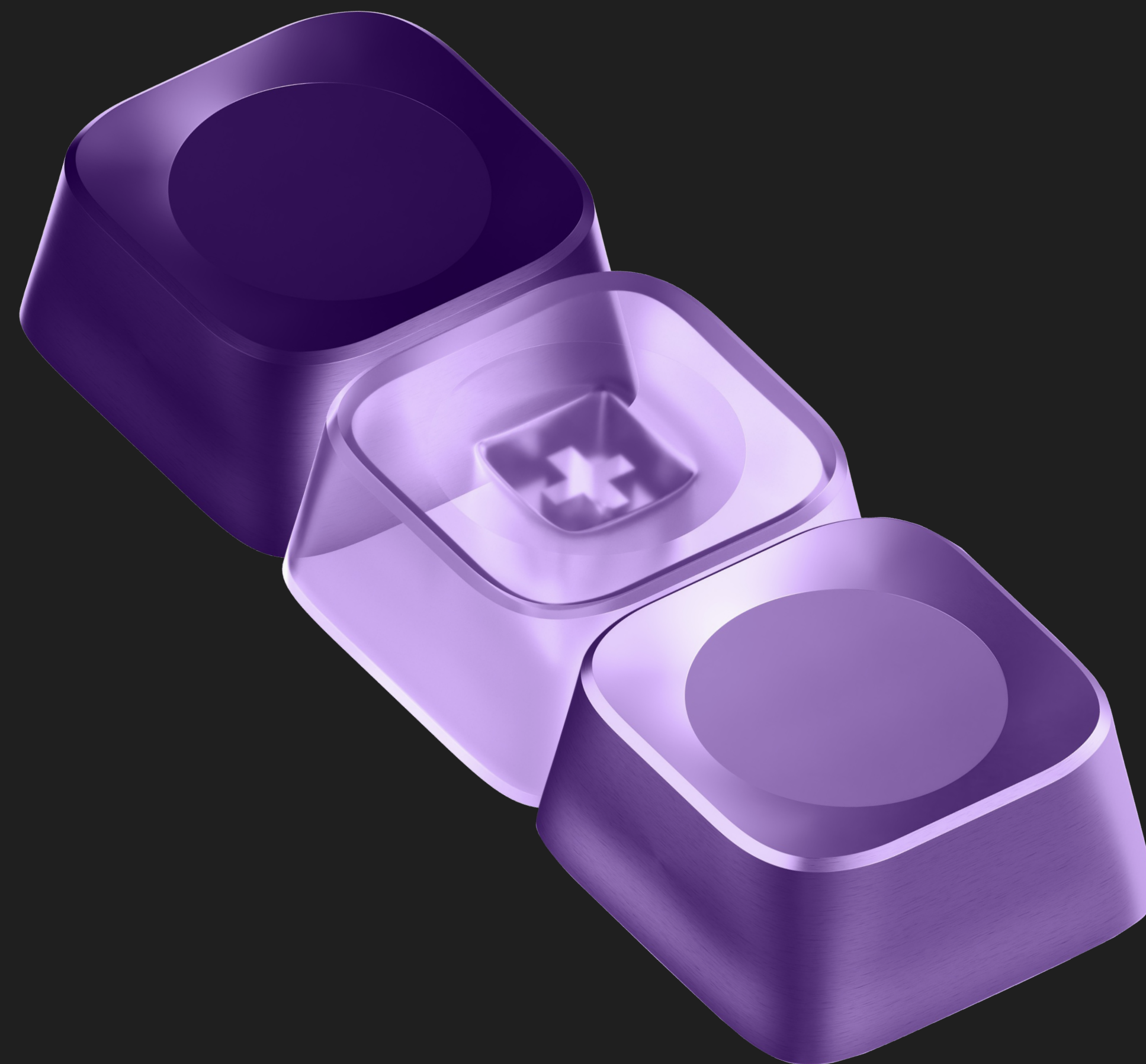


Резюме

- Изоляция (MVCC): снимки (Snapshots) + xmin/xmax + xip_list = отсутствие блокировок, но плата за это — мусор в памяти и на диске.
- Надёжность (WAL): Append-only log + fsync = гарантия сохранности, но плата за это — двойная запись (в WAL, а потом через Checkpointer в файлы данных).
- СУБД — это сложная машина конечных состояний, где каждый выигрыш в одном месте стоит ресурсов в другом.
- Понимание абстракций (аномалии, уровни изоляции) позволяет писать корректный код.
- Понимание физики (MVCC, WAL, Буферы) позволяет писать быстрый код и спасать production.



Вопросы



Минутка обратной связи

